

Tarea 4

Sistemas de Turnos Médicos

Josefa Fernández, Felipe Fuentes, Martin Garrido

Grupo Web JFM

Introducción General de la aplicación (1min)

Nombre Aplicación:
Sistemas de Turnos Médicos.

Propósito Principal:
Nuestro proyecto es una aplicación web integral diseñada para simplificar y optimizar la gestión de citas médicas. Permite a los pacientes reservar y administrar sus turnos de forma sencilla, mientras que los médicos pueden gestionar eficientemente sus horarios y la disponibilidad.



Introducción General de la aplicación (1min)

Funcionalidades Más Relevantes:

Para Pacientes	Para Médicos	Para Administradores
Acceso rápido y seguro	Gestión de agenda	Supervisión completa del sistema
Agendamiento y cancelación de citas	Confirmación/rechazo de citas	Gestión de usuarios y citas
Visualización del historial	Acceso al historial de pacientes	

Decisiones arquitectonicas (2-3 mins.):

Patrones de Renderizado	Uso	Qué problema resuelve
SSG	Páginas con contenido estático o cambian poco (**Landing Page**).	Pre-renderizado en tiempo de construcción, resulta en una carga inicial rápida, mejora el SEO y reduce la carga del servidor. Ideal para contenido que no necesita ser fresco en cada solicitud.
ISR	Páginas que necesitan actualizarse periódicamente, pero no en cada solicitud (**Perfiles Médicos**).	Regenerar páginas estáticas en segundo plano a intervalos definidos, asegura que la información no esté demasiado desactualizada sin necesidad de un nuevo despliegue completo.
SSR	Páginas de atos actualizados en cada solicitud y contenido personalizado (**Panel de Administrador, Dashboards de Paciente/Doctor**).	Usuarios ven la información más reciente y personalizada al renderizar la página en el servidor para cada solicitud. Crucial para dashboards dinámicos y contenido protegido por autenticación.
Server Components	Componentes en partes de la aplicación donde la interactividad del cliente no es necesaria.	Minimiza significativamente la carga de JavaScript en el cliente, lo que mejora los tiempos de carga inicial y el rendimiento general de la aplicación.

Decisiones arquitectonicas (2-3 mins.):

Elección base de datos

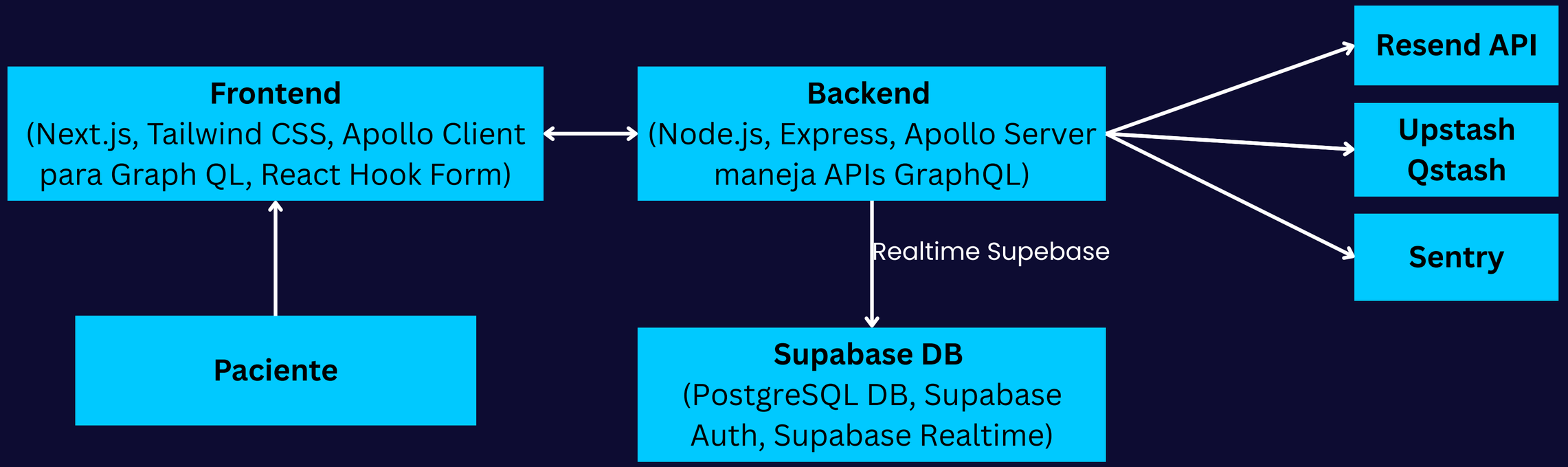
- **Supabase** nos proporcionó:
  - una base de datos PostgreSQL robusta y escalable
  - un sistema de autenticación seguro (Supabase **Auth**)
  - capacidades en tiempo real (Supabase **Realtime**).
  - No hay complejidad de gestionar infraestructura de back-end desde cero.
- **GraphQL con Apollo Server:**
  - Implementamos un servidor GraphQL en el backend (Node.js + Express + Apollo Server), permitiendo a los clientes solicitar exactamente los datos que necesitan.

Solución Tarea 3:

- Los datos de usuario y las imágenes se almacenan en **Firebase Authentication y Firestore.**

Decisiones arquitectonicas (2-3 mins.):

Diagrama Arquitectura General de la Solución



Flujo de Datos: Las solicitudes del usuario inician en el Frontend, pasan por el Backend , interactúan con Supabase para datos y autenticación, y pueden activar procesos asíncronos o notificaciones a través de los servicios externos.

Decisiones arquitectonicas (2-3 mins.):

Frontend
Paciente navega agendamiento citas

Frontend consulta disponibilidad médicos
Realiza consulta GraphQL al backend para obtenerlo.

Backend (Apollo Server)
recibe consulta.

Backend obtiene disponibilidad
Interactúa con Supabase y da lo disponible al frontend

Paciente (Frontend)
Selecciona hora, doctor y envía mutation GraphQL para agendar cita

Backend
Valida y actualiza tabla appointments en Supabase

Procesos Asíncronos (Backend)
Backend envia mensaje a cola en Upstash Queue para generar PDF y enviar mail de confirmación al paciente por Resend API

Supabase Realtime
Detecta el cambio en disponibilidad de doctor y notifica a los clientes suscritos, actualizando UI.

Decisiones arquitectonicas (2-3 mins.):

Patrón principal utilizado: Actualizaciones en Tiempo Real con Supabase Realtime

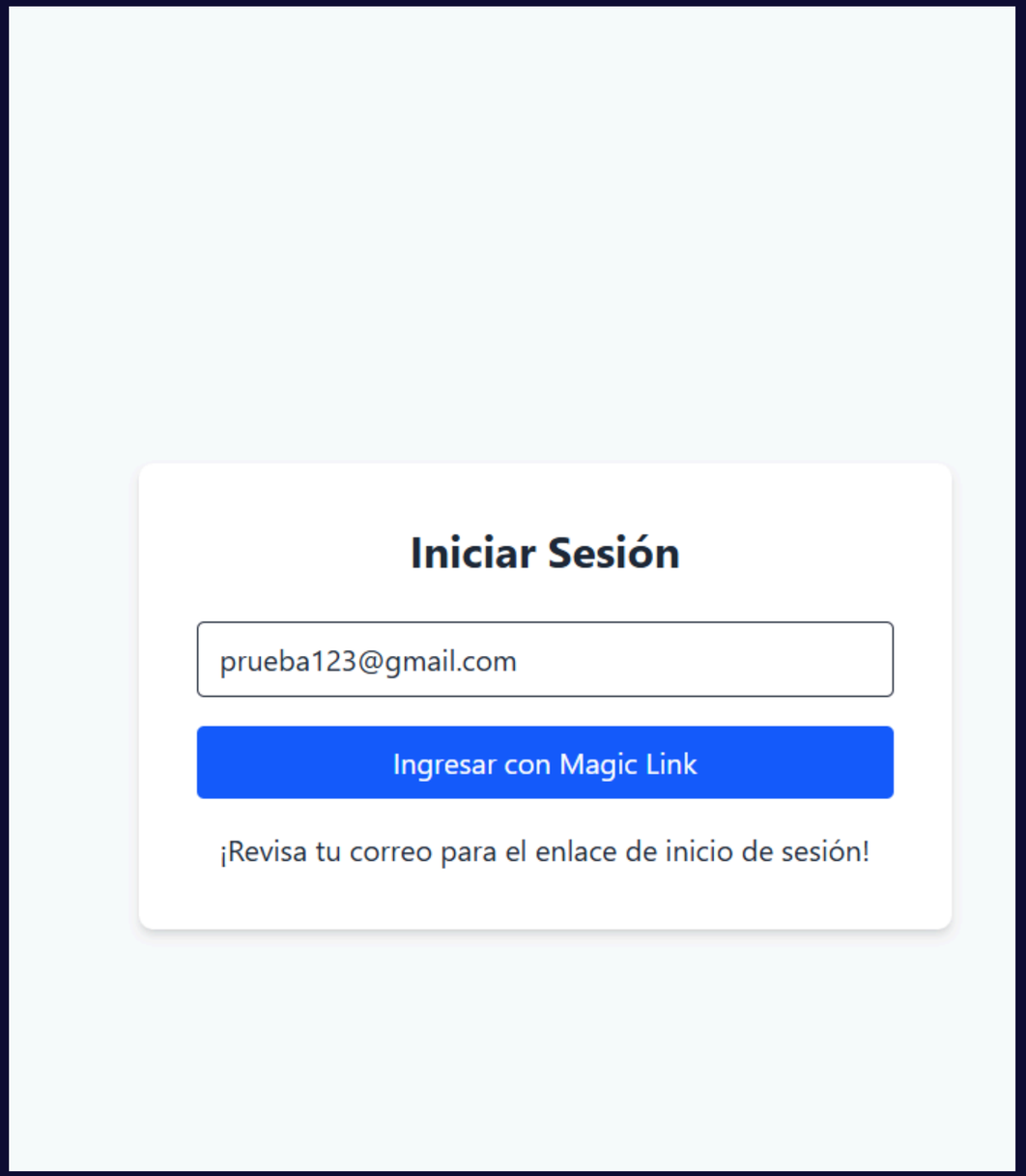
Como funciona:
Supabase Realtime permite a la aplicación suscribirse a cambios en la base de datos. Cuando un dato (ej., el estado de una cita o la disponibilidad de un doctor) se modifica, Supabase envía automáticamente una notificación a todos los clientes suscritos.

Beneficios:
Permite una experiencia de usuario fluida y dinámica, donde la información se actualiza instantáneamente. Esto esimportante , ya que los cambios en la disponibilidad o las citas deben reflejarse de inmediato para evitar conflictos y mejorar la comunicación.

Implementacion de bonuses (0-2 mins.)

Zero- password Architecture

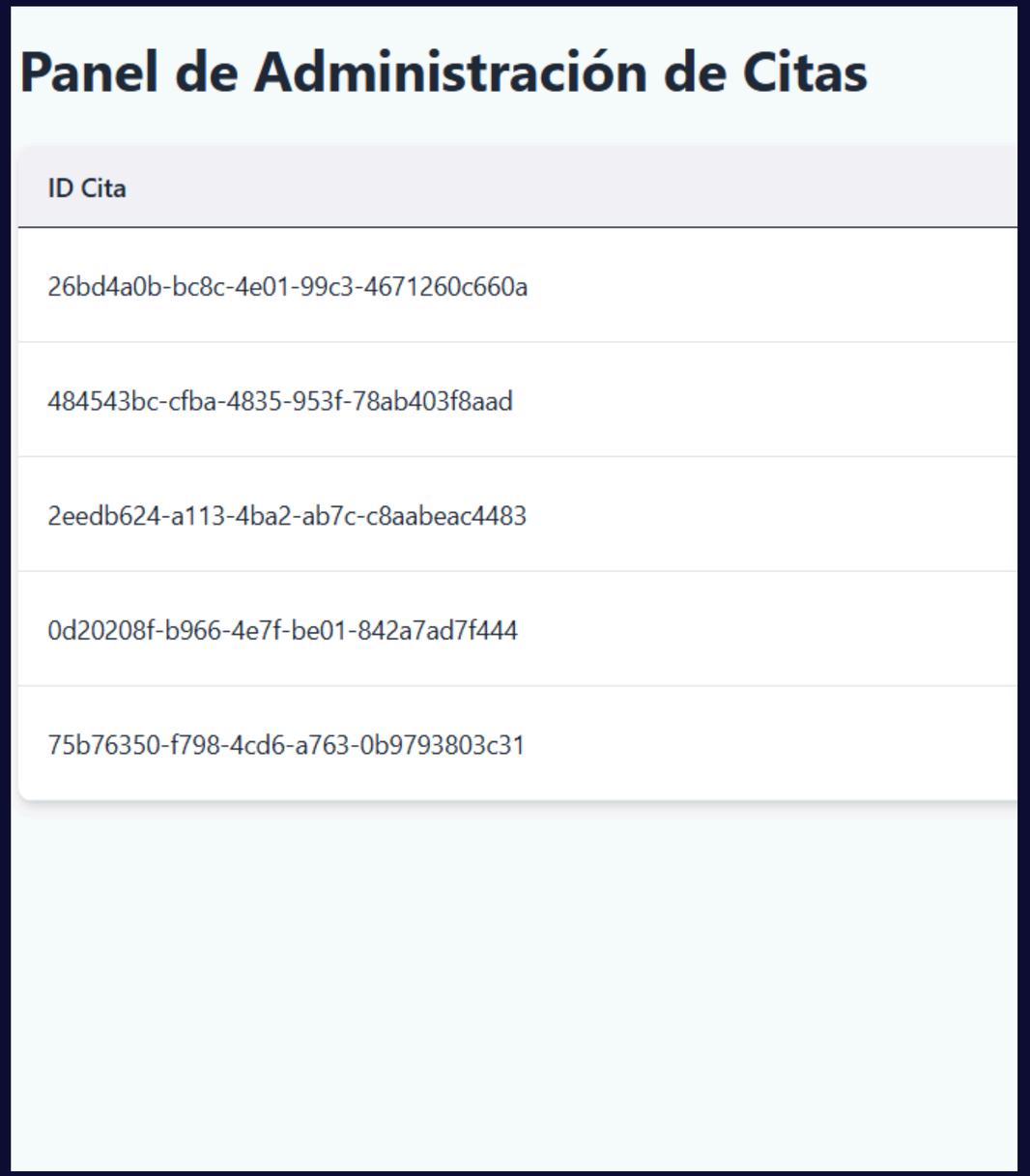
- Autenticación seguro sin contraseña.
- Página de inicio de sesión mostrando la opción de Magic Link.



Bonus

Implementación de roles de usuario

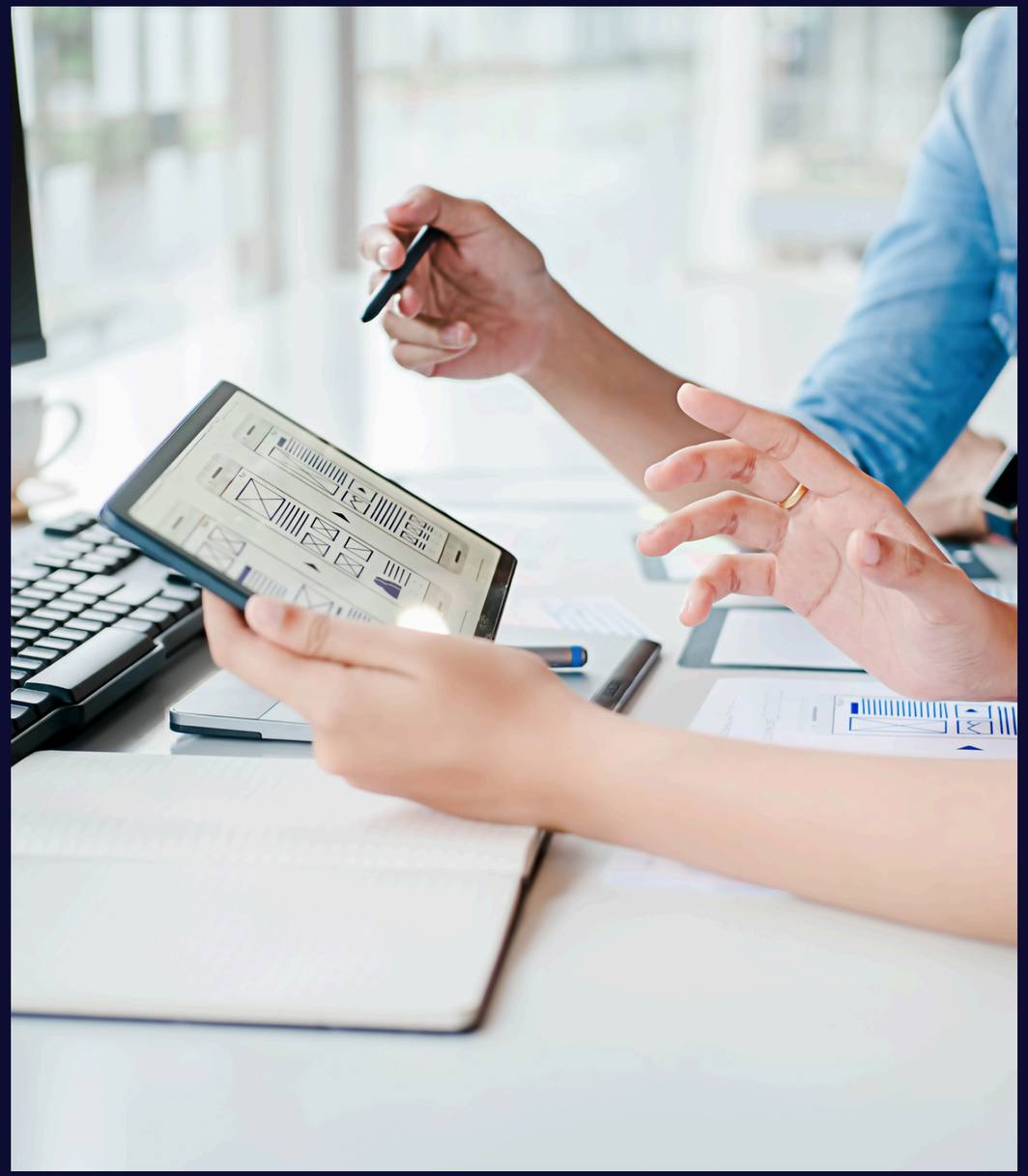
- Roles de usuario uso de Supabase Auth con RLS
- Paciente, admin, doctor.



Bonus

No usar CSR

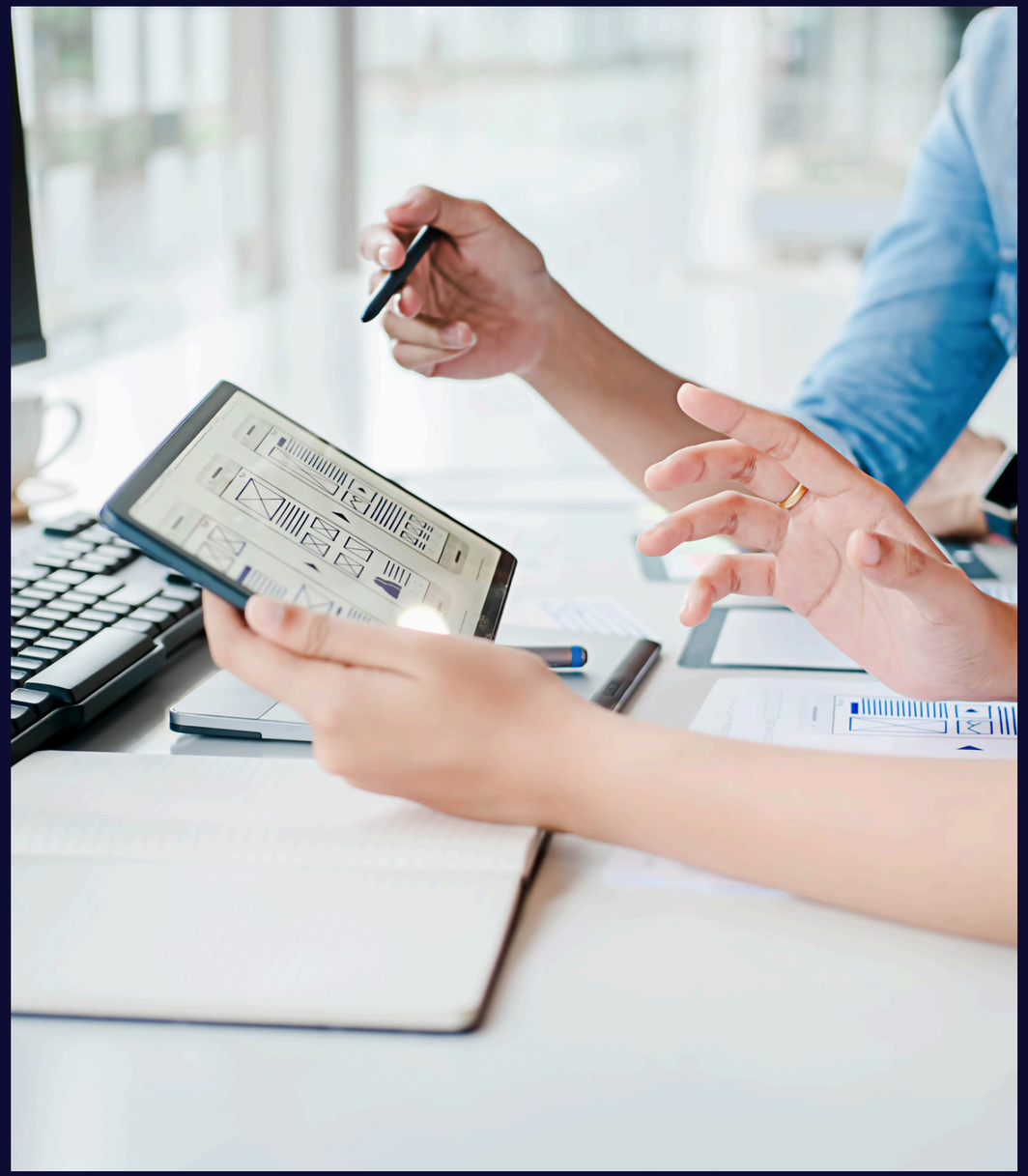
- Priorización del uso de Server Components para minimizar la carga de JavaScript en el cliente, mejorando los tiempos de carga y el SEO.
- Código: Ejemplos de componentes que no tienen la directiva "use client" y que realizan fetch de datos directamente en el servidor.



Bonus

Cola de Procesamiento

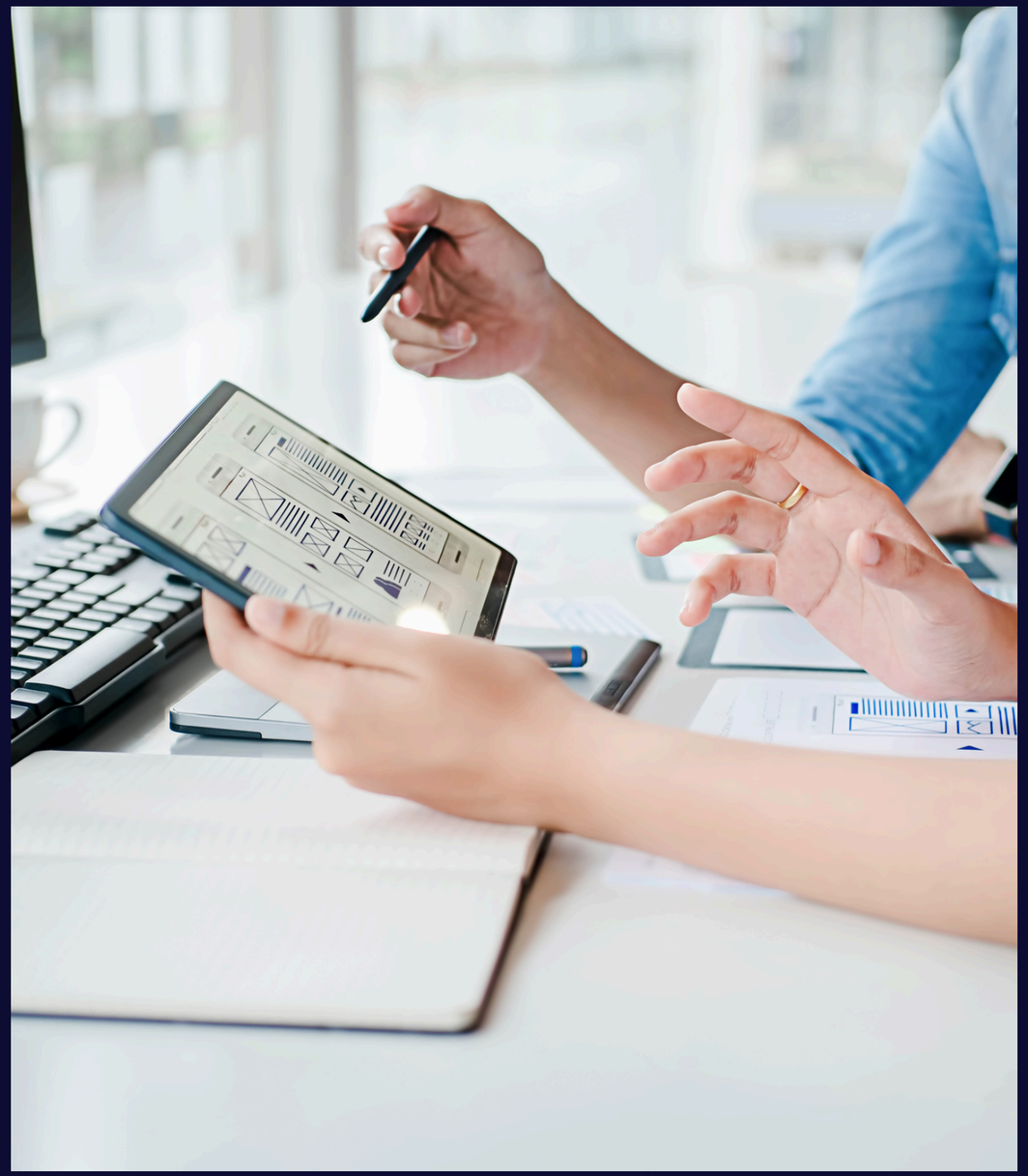
- Gestión de procesos asíncronos como la generación automática de PDFs de confirmación de cita y su envío por correo electrónico.
- Upstash Dasboard, dashboard Upstash Qtash mostrando mensjaes de cola o trabajos completados o c´odigp del backend que encola un mensaje en Qstash o que utiliza la API de Resend.



Bonus

Implementar GraphQL

- API GraphQL completa con queries, mutations y subscriptions correspondientes, incluyendo optimizacion de queries y manejo de errores



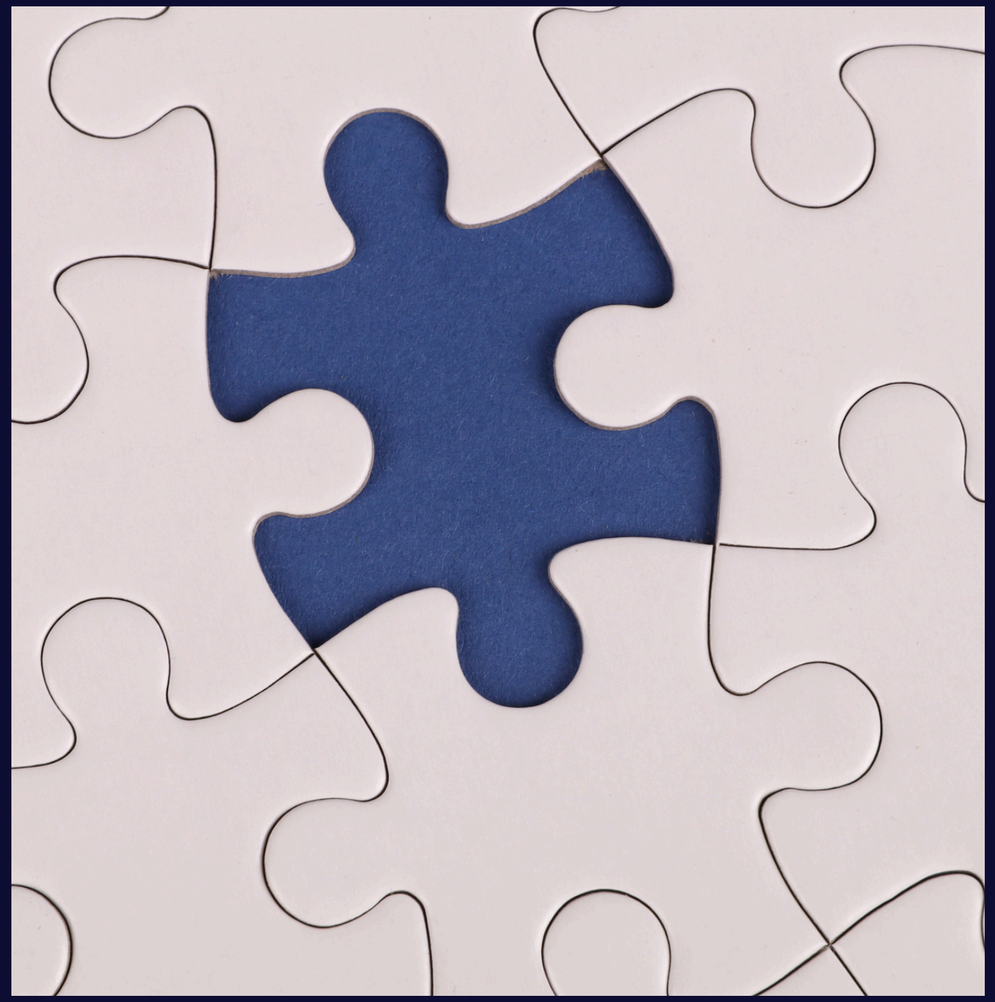
Implementacion de bonuses (0-2 mins.)

Principal Obstáculo Técnico

- Integrar Supabase Auth / Roles.
- Sincronizar autenticación cliente/servidor.
- Aplicar autorización consistente (RLS)

La Solución Implementada:

1. **Políticas de Row Level Security (RLS) en Supabase:** Esto asegura que, incluso si hay una brecha en la capa de aplicación, la base de datos misma haga cumplir los permisos de lectura/escritura según el rol del usuario, añadiendo una capa crítica de seguridad.
2. **Manejo de Redirecciones en middleware.ts:** El middleware.ts de Next.js fue configurado para interceptar todas las solicitudes, verificar el token de sesión de Supabase y el rol del usuario, y redirigir a rutas no autorizadas o a la página de inicio de sesión cuando fuera necesario. Esto creó una barrera de seguridad efectiva antes de que las solicitudes llegaran a las páginas.



Hallazgos tecnicos y aprendizajes (2 mins.):

Insights Técnicos Valiosos para el Curso

1. **La Potencia de un BaaS (Backend as a Service) con RLS:**
    a. **Hallazgo**: BaaS coomo Supabase con PosstgreSQL + Row Level Security: Simplifica backend: la gestión de la base de datos, la autenticación y la autorización a nivel de datos.
    b. **Aprendizaje Clave:** Permite enfocarse en la lógica pero requiere comprender como usar Supabase y seguridad RLS.

2. **Server Componentes en Next.js**
    a. **Hallazgo**: La arquitectura de Server Components en Next.js (y la diferenciación entre "use client" y componentes de servidor por defecto.
    b. **Aprendizaje Clave:** Los Server Components reduce la cantidad de JavaScript enviada al navegador, lo que resulta en tiempos de carga más rápidos

Hallazgos tecnicos y aprendizajes (2 mins.):

Qué Haríamos Diferente si Volviéramos a Empezar el Proyecto:

- Integrar características tiempo real antes
- Automatizar CI/CD desde el día uno.
- Planficar mejor nuestros tiempos

Muchas Gracias

Autenticación: Arquitectura Zero-Password

"Implementamos autenticación 100% sin contraseñas usando Supabase Auth."

Stack:

- Next.js 15 + Supabase Auth
- Magic Links por email únicamente
- Middleware automático de sesiones

Bonus Implementados:

✅ Zero-Password Architecture (5%) – Solo Magic Links
✅ Next.js Server Components – Optimización de rendimiento
✅ Real-time Ready – Supabase preparado para tiempo real

Código Principal – AuthProvider

◆ Gestión de Estado Global con React + Supabase

- Se crea un cliente Supabase en el cliente (createClientComponentClient).
- Se manejan los estados user, session y loading.
- useEffect escucha los cambios de sesión con onAuthStateChange.
- Al cargar, se obtiene la sesión activa con getSession.
- El contexto se expone solo cuando loading es false.

```
export function AuthProvider({ children }: { children: React.ReactNode }) {
  const supabase = createClientComponentClient();
  const [user, setUser] = useState<User | null>(null);
  const [session, setSession] = useState<Session | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const { data: { subscription } } = supabase.auth.onAuthStateChange(
      (_event, session) => {
        setSession(session);
        setUser(session?.user ?? null);
        setLoading(false);
      }
    );

    const getInitialSession = async () => {
      const { data: { session } } = await supabase.auth.getSession();
      setSession(session);
      setUser(session?.user ?? null);
      setLoading(false);
    };

    getInitialSession();
    return () => subscription?.unsubscribe();
  }, [supabase.auth]);

  return (
    <AuthContext.Provider value={{ user, session, loading }}>
      {!loading && children}
    </AuthContext.Provider>
  );
}
```

Flujo Técnico Completo

🔹 Login → Callback → Dashboard/

```
/ Login con Magic Link
await supabase.auth.signInWithOtp({
  email,
  options: { emailRedirectTo:
`${location.origin}/auth/callback` },
});

// Callback automático
export async function GET(req: NextRequest) {
  const code = req.nextUrl.searchParams.get('code');
  if (code) await
supabase.auth.exchangeCodeForSession(code);
  return NextResponse.redirect('/dashboard');
}

// Protección de rutas
const { user, loading } = useAuth();
if (!user) return <div>Debes iniciar sesión</div>;
```

🔹 Middleware Automático

```
export async function middleware(req: NextRequest) {
  const supabase = createMiddlewareClient({ req, res });
  await supabase.auth.getSession(); // Auto-refresh
  return res;
}
```

Resultados y Ventajas

Texto para una casilla:

🎯 Logros Técnicos:

- 0 campos de contraseña en toda la app
- Sesiones automáticas que no expiran
- Tipado completo con TypeScript
- Integración GraphQL con user.id

🔒 Beneficios de Seguridad:

- ✅ Anti-phishing: enlaces únicos temporales
- ✅ No password leaks: sin credenciales almacenadas
- ✅ JWT tokens con Supabase

⚡ Performance:

- Server Components = menos JS al cliente
- Middleware eficiente con auto-refresh
- Un solo método de auth = simplicidad arquitectural